

 Institución Universitaria Acreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Display multiplexado y conversión binario a BCD
Código de la guía (No.):	G3EDL
Taller(es) o Laboratorio(s) aplicable(s):	Microelectrónica y Nanotecnología
Tiempo de trabajo práctico estimado:	4 horas
Asignatura(s) aplicable(s):	Electrónica Digital y Laboratorio
Programa(s) Académico(s) / Facultad(es):	Tecnología en Automatización Electrónica - Tecnología en Gestión de Redes de Telecomunicaciones

COMPETENCIA	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Diseña y analiza circuitos lógicos combinacionales y secuenciales para la resolución de problemas que involucren circuitos electrónicos digitales, empleando metodologías de diseño digital con la ayuda de herramientas computacionales de desarrollo y simulación.	Circuitos digitales para el manejo de display de siete segmentos multiplexado – Conversión numérica binaria a BCD (Binary Coded Decimal).	Implementación sobre FPGA descrito en VHDL de circuitos digitales combinacionales para la presentación de resultados en un sistema multiplexado de 8 displays de siete segmentos.

2. FUNDAMENTO TEÓRICO

La forma más simple de mostrar resultados generados por un circuito digital es utilizando LEDs¹ o displays. Con LEDs solo es posible presentar los resultados bit a bit, mientras que con displays un grupo de bits adquiere un significado que permite representar símbolos más amigables para las personas. Los displays más comunes son el LCD² y el de siete segmentos. En este último cada uno de los siete segmentos del display internamente tiene un LED y está nombrado con una letra (de la *a* hasta la *g*) como puede verse en la Figura 1 [1]. Las configuraciones típicas de los displays son ánodo común y cátodo común. En el primero todos los LEDs del display están unidos entre sí por medio de sus ánodos, mientras que en el segundo la unión se hace a través de sus cátodos. La Nexys 4 y la Nexys A7 cuentan con un arreglo de ocho displays de ánodo común [2] [3].

La combinación de los 7 LEDs permite generar hasta $2^7 = 128$ símbolos diferentes, entre ellos los dígitos del sistema numérico decimal como puede verse en la Figura 2. El LED asociado a cada segmento enciende (ON) cuando hay una diferencia de potencial positiva entre el ánodo y el cátodo, en caso contrario no enciende (OFF). Por lo tanto, si queremos llevar un segmento a ON debemos poner un 0 lógico en el cátodo y un 1 lógico en el ánodo. Pero debido a que los ánodos en estos kits de desarrollo están conectados a través de transistores, para dar mayor

¹Light Emitting Diode

²Liquid Crystal Display

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

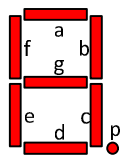


Figura 1: Identificación de los segmentos de un display de siete segmentos.

corriente, la señal de activación del ánodo es invertida. Por ende, para llevar un LED a ON debe conectarse el ánodo del display a θ lógico y el correspondiente cátodo también a θ lógico.

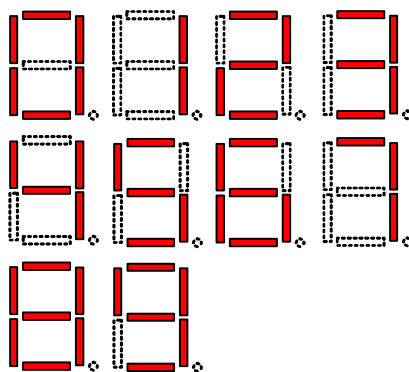


Figura 2: Diez patrones de iluminación en un display de siete segmentos para generar los dígitos decimales.

Displays multiplexados

Los displays de siete segmentos de los kits de desarrollo de FPGA del Laboratorio tienen una característica muy importante, y es que son multiplexados [2] [3]. Esto quiere decir que no se pueden mostrar datos diferentes de manera simultánea en varios displays, ya que los cátodos de los segmentos correspondientes en cada display están conectados entre sí. Por lo tanto, solo existe un cátodo para todos los LEDs del segmentos *a*, denominado *CA*, que permite poner en ON-OFF en forma simultánea todos los segmentos *a* de los ocho displays. De igual manera existen los cátodos *CB*, *CC*, *CD*, *CE*, *CF*, *CG* y *DP* para llevar a ON-OFF los segmentos *b*, *c*, *d*, *e*, *f*, *g* y el punto *p* del display respectivamente (véase Figura 3). La dinámica consiste en simultáneamente activar el display en el que se quiere encender los segmentos, poniendo un θ lógico en su ánodo, y encender los LEDs de los segmentos que forman el símbolo que se quiere visualizar, poniendo un θ lógico en el cátodo correspondiente. Por ejemplo, si queremos mostrar el número 4 solo en el display 6 se debe poner θ lógico al ánodo *AN6* y 1 lógico en los demás ánodos, y simultáneamente poner θ lógico en los cátodos *b*, *c*, *f* y *g*, y 1 lógico en los demás.

Para utilizar apropiadamente el sistema multiplexado de displays es necesario controlar la activación de sus ánodos. Esto se puede hacer activando uno a uno de manera secuencial por medio de un decodificador de 3 a 8 con salidas activas en bajo. Las entradas del decodificador se conectan a un contador de 3 bits, cuenta de 0 a 7, a una frecuencia de 400 *Hz* para activar 50 veces por segundo cada uno de los ocho displays. De esta manera, lo que se escriba en cada display el ojo humano lo verá 50 veces durante un segundo, y de esta forma se le puede “engañar” aparentando que lo escrito en cada display está siempre allí. Ya que solo hay un cátodo *a*, un cátodo *b*, etc., solo se requiere un convertidor de BCD a 7 segmentos, pero como son ocho displays diferentes en los cuales escribir, los datos para cada display deben llevarse a

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

un multiplexor 8 a 1 de 4 bits, donde las líneas de selección sean los mismos 3 bits del contador que cuenta a una frecuencia de 400 Hz , de esta forma ánodo activo y dígito a mostrar en el display estarán sincronizados (véase Figura 4).

En la Figura 4 pueden observarse el multiplexor 4 a 1 de 4 bits, el cual se puede describir con la sentencia *wit-select-when* de VHDL, el decodificador de 3 a 8 con salidas activas en bajo, el cual se puede describir con la sentencia *when-else* de VHDL, y el convertidor de código de BCD a 7 segmentos que igualmente se puede describir con la sentencia *when-else*.

Los bloques con sombreado gris son un generador de reloj de 400 Hz y un contador de 3 bits que son sistemas secuenciales y que en esta práctica simplemente los usaremos para nuestro propósito, pero serán estudiados en posteriores guías. El módulo de reloj a utilizar puede verse en el Código 2.1 y el contador de 3 bits en Código 2.2.

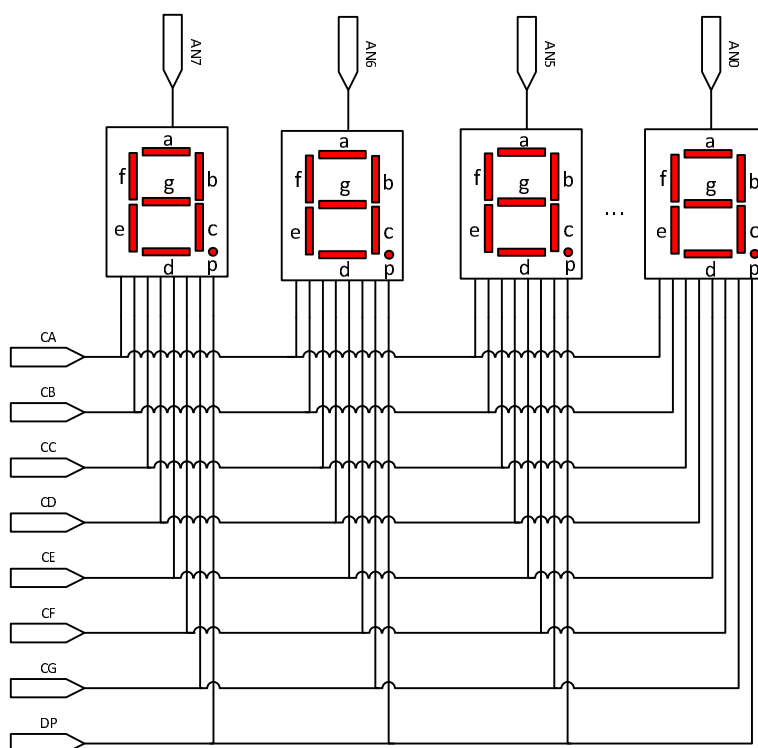


Figura 3: Estructura de conexión del arreglo de ocho displays en la Nexys 4 y la Nexys A77.

Al multiplexor 8 a 1 ingresan números binarios de 4 bits que secuencialmente pasarán al convertidor de código BCD a 7 segmentos, si el binario de 4 bits representa un número BCD [4], esto es, es un número del 0 a 9, se mostrará en el display correspondiente, de otra forma se encenderá solo el punto del display.

Convertidor de números binarios a BCD

Un método para convertir números binarios a BCD³ es empleando el conocido algoritmo *double dabble* o *shift-and-add-3* [5]. En general, un número binario de n bits producirá un

³ Los números BCD son la representación en binario, con 4 bits, de los dígitos del sistema numérico decimal, esto es, los números de 0 a 9.

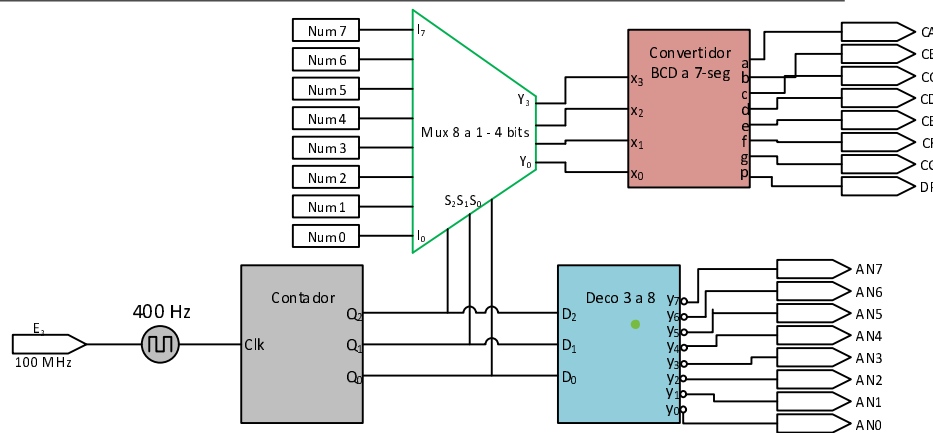


Figura 4: Diagrama de bloques para la sincronización de los displays en el sistema multiplexado de la Nexys 4 y la Nexys A7.

número BCD de $4 \times \text{ceil}(n/3)^4$ bits, donde cada grupo de 4 bits representa un dígito BCD.

El algoritmo *double dabble* requiere, para convertir un número binario de n bits a BCD, un registro que almacena tanto el número binario como todos los dígitos del número BCD, esto es, un registro de tamaño $n + 4 \times \text{ceil}(n/3)$ bits es suficiente. En este registro los n bits de la derecha serán el número binario a convertir, mientras que los $4 \times \text{ceil}(n/3)$ bits de la izquierda serán el número BCD del resultado. Por ejemplo, para convertir un número binario de 8 bits a BCD, el número BCD resultante requiere $4 \times \text{ceil}(8/3) = 12$ bits, por lo tanto, el algoritmo *double dabble* necesita un registro de $8 + 12 = 20$ bits. Esta expresión no optimiza el tamaño del registro donde se almacenan los datos que arrojan el algoritmo, por ejemplo, números binarios de 13 bits representan números decimales en el rango $[0, 2^{13} - 1] = [0, 8191]$, esto es, cuatro dígitos BCD (16 bits), mientras que la expresión arroja $4 * \text{ceil}(13/3) = 20$ bits. Sin embargo, la expresión cumple hasta números de 12 bits.

La Tabla 1 muestra el paso a paso de la conversión del número de 8 bits 10010110_2 a BCD. Los 8 bits de menor peso son el número binario a convertir y los dígitos BCD, grupos de 4 bits que inicializan todos en 0, de derecha a izquierda son las unidades, decenas y centenas respectivamente. Este algoritmo consiste en verificar si cada uno de los dígitos BCD que se van generando es mayor a 4, en cuyo caso de ser cierto, se le suma 3 y posteriormente se desplaza una posición a la izquierda. Esto garantiza que, por ejemplo, el $12_{10} = 1100_2$ se convierta en $0001\ 0010$. Esta iteración se realiza un número de veces igual al número de bits del número a convertir, 8 en este caso.

El diagrama de flujo del algoritmo de conversión de un número binario de 13 bits a 4 dígitos BCD puede verse en la Figura 5. Este algoritmo funciona para números binarios de 13 bits con el cual se pueden generar números hasta el 8191 ($2^{13} - 1$). Se declara una variable numérica no signada (Z) de tamaño igual al tamaño del número binario más el tamaño que ocupan todos los dígitos BCD que se van a generar. En este caso, el número binario (bin) tiene una longitud ($Length_{bin}$) de 13 bits, mientras que los dígitos BCD serán 4 (cada uno de 4 bits), esto es, 16 bits. Por lo tanto la variable Z debe tener una longitud de 29 bits. Posteriormente, se inicializa Z a 0 y se carga el número binario, bin , en los 13 bits de menor peso de Z ($Z[12:0]$). A partir de acá se inicia un ciclo, que continuará por 13 veces, que verifica si cada uno de los dígitos BCD (unidades - BCD_u , decenas - BCD_d , centenas - BCD_c y unidades de mil -

⁴ La función *ceil* es la aproximación por encima del resultado de una operación, por ejemplo, $\text{ceil}(8/3) = 2,33 \rightarrow 3$.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- Declaración de entidad
entity modClock is
    Port ( fastClock : in STD_LOGIC;
          slowClock : inout STD_LOGIC;
          prescaler : in INTEGER RANGE 0 TO 50000000);
end modClock;

-- Definición de la arquitectura
architecture Behavioral of modClock is
begin

slowClockProcess:process(fastClock)
    variable countFastCycles:INTEGER RANGE 0 TO 50000000;
begin
    if rising_edge(fastClock) then
        countFastCycles := countFastCycles + 1;
        if countFastCycles = prescaler then
            slowClock <= NOT slowClock;
            countFastCycles := 0;
        end if;
    end if;
end process slowClockProcess;

end Behavioral;

```

Código 2.1: Módulo VHDL para generar relojes de baja frecuencia a partir de un reloj de 100 MHz.

Tabla 1: Proceso paso a paso de la conversión de $10010110_2 = 150_{10}$ a BCD.

Centenas	Decenas	Unidades	Binario	Comentario
0000	0000	0000	10010110	$C - U - D < 5, \leftarrow 1$
0000	0000	0001	00101100	$C - D - U < 5, \leftarrow 2$
0000	0000	0010	01011000	$C - D - U < 5, \leftarrow 3$
0000	0000	0100	10110000	$C - D - U < 5, \leftarrow 4$
0000	0000	1001	01100000	$U > 4, +3, \leftarrow 5$
0000	0001	1000	11000000	$U > 4, +3, \leftarrow 6$
0000	0011	0111	10000000	$U > 4, +3, \leftarrow 7$
0000	0111	0101	00000000	$D - U > 4, +3, \leftarrow 8$
0001	0101	0000	00000000	Resultado = 150

BCD_uM) es mayor a 4, en caso afirmativo le suma 3 y en caso negativo deja el dígito intacto, y posteriormente desplaza la variable completa, Z, una posición a la izquierda. Finalmente, las unidades de mil, las centenas, las decenas y las unidades del número BCD se encuentran en la variable Z en las posiciones 28:25, 24:21, 20:17 y 16:13 respectivamente.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
-- Declaración de entidad
entity eCounter3Bit is
    Port ( Clk : in STD_LOGIC;
          Q : inout STD_LOGIC_VECTOR (2 downto 0) );
end eCounter3Bit;

-- Definición de la arquitectura
architecture Behavioral of eCounter3Bit is
    signal sQ : UNSIGNED (2 downto 0) := (others => '0'); -- := "000"
begin

    process(Clk)
    begin
        if rising_edge(Clk) then
            if (sQ = 7) then
                sQ <= (others => '0');
            else
                sQ <= sQ + 1;
            end if;
        end if;
    end process;
    Q <= STD_LOGIC_VECTOR (sQ);

end Behavioral;

```

Código 2.2: Módulo para generar un contador de 3 bits ascendente de 0 a 7.

3. OBJETIVO(S)

- Comprender y utilizar el sistema multiplexado de displays de 7 segmentos de los sistemas de desarrollo Nexys 4 y Nexys A7.
- Comprender e implementar los convertidores de código BCD a 7 segmentos.
- Implementar en hardware un algoritmo de conversión de números binarios a BCD.

4. RECURSOS REQUERIDOS

- Estación de trabajo.
- Vivado Design Suite.
- Nexys 4 o Nexys A7.

 ITM Institución Universitaria Acreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

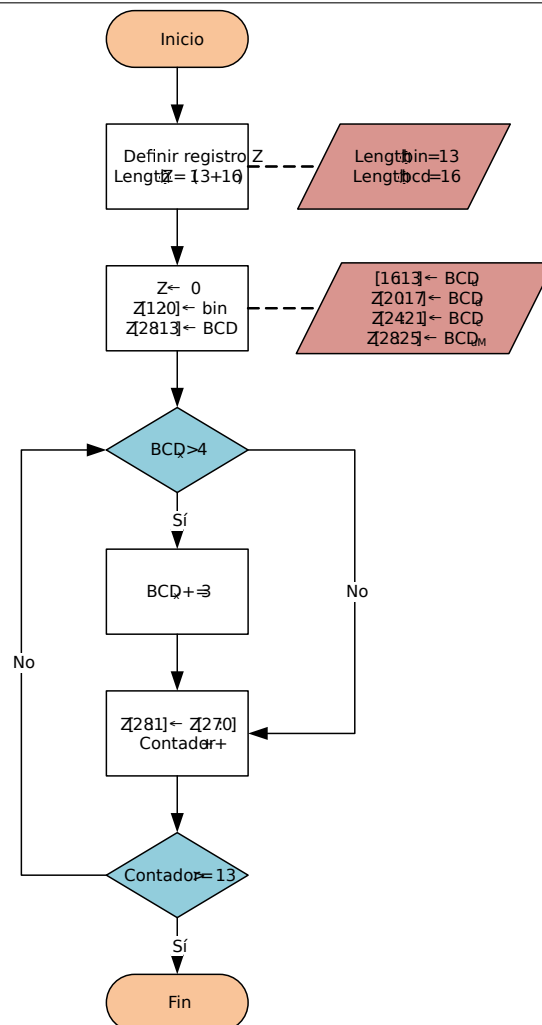


Figura 5: Diagrama de flujo de convertidor de número binario de 13 bits a BCD.

5. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

Describir, simular e implementar un circuito digital, sobre la Nexys 4 o la Nexys A7, al cual se le ingrese un número binario de 13 bits, por medio de switches, y muestre en cuatro displays de 7 segmentos el número decimal equivalente. El diagrama de bloques del sistema puede verse en la Figura 6, en el cual se observan los puertos de entrada, el número binario de 13 bits y el reloj de 100 MHz, los puertos de salida, los 8 cátodos y los 8 ánodos para los displays de 7 segmentos, y tres señales, *sClk400Hz*, *sQ* y *numBCD*, empleadas para conectar los módulos de reloj con el contador de 3 bits, el contador con el decodificador y el multiplexor y el multiplexor con el convertidor BCD a 7 segmentos respectivamente.

El módulo de reloj, véase Código 2.1, y el módulo contador de 3 bits, véase Código 2.2, se deben instanciar en el módulo integrador como se ve en Código 5.1. El entero 125000 en la instancia de *modClock* es para generar un reloj de 400 Hz y poder encender y apagar cada uno de los 8 displays 50 veces por segundo.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

Para nuestro propósito en la entradas *Num7*, *Num6*, *Num5* y *Num4* pondremos números binarios de 4 bits que no sean BCD, esto es, del binario 10 al 15 (*1010*, *1011*, *1100*, *1101*, *1110* o *1111*), mientras que las entradas *Num3*, *Num2*, *Num1* y *Num0* serán los dígitos BCD resultantes de la conversión binario a BCD de 13 bits.

La simulación debe hacerse por medio de un archivo testbench.

Todos los grupos deben implementar el mismo circuito.

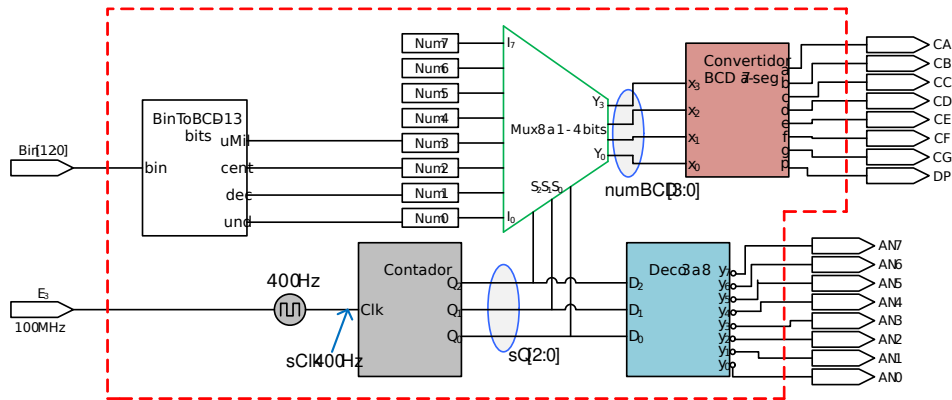


Figura 6: Diagrama de bloques del convertidor binario a BCD de 13 bits con visualización en displays de 7 segmentos.

6. PARÁMETROS PARA ELABORACIÓN DEL INFORME

Si la práctica requiere la presentación de un informe el docente facilitará una plantilla en Word para escribirlo. Si los estudiantes lo desean pueden utilizar $\text{\LaTeX}$ empleando los mismos parámetros de la plantilla propuesta.

7. DISPOSICIÓN DE RESIDUOS

Esta práctica no genera residuos.

Referencias

- [1] W. Storr, "7-segment display and driving a 7-segment display," Aug 2022.
- [2] Digilent, "Nexys A7 FPGA Board Reference Manual," tech. rep., Digilent, Inc., Pullman, WA, 2019.
- [3] Digilent, "Nexys 4 FPGA Board Reference Manual," tech. rep., Digilent, Inc., Pullman, WA, 2016.
- [4] M. Rafiquzzaman and S. A. McNinch, *Digital Logic with an introduction to Verilog and FPGA-based design*. Hoboken, NJ: Wiley, 2019.
- [5] C. Falconer, "An explanation of the double-dabble bin-bcd conversion algorithm," 2016.

 Institución Universitaria Acreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	08-10-2018

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity SistConvBinToBCD is
    Port ( Clk100Mhz : in STD_LOGIC;
          Bin : in STD_LOGIC_VECTOR (12 downto 0);
          An : out STD_LOGIC_VECTOR (7 downto 0);
          Ka : out STD_LOGIC_VECTOR (7 downto 0));
end SistConvBinToBCD;

architecture Behavioral of SistConvBinToBCD is
    -- Declaración de los componentes

    signal sQ : STD_LOGIC_VECTOR (2 downto 0);
    signal numBCD : STD_LOGIC_VECTOR (3 downto 0);
    signal sClk400Hz : STD_LOGIC;

begin

    i0_modClock:modClock port map(fastClk => Clk100MHz,
                                  slowClk => sClk400Hz, prescaler => 125000);
    i0_eCounter3bit:eCounter3bit port map(Clk => sClk400Hz, Q => sQ);

end Behavioral;

```

Código 5.1: Módulo integrador con las instancias del reloj de 400 *Hz* y el contador de 3 bits.

Elaborado por:	Sergio Ignacio Serna Garcés, Ph.D.
Revisado por:	Andrés Felipe Tobón Mejía, M.Sc.
Versión:	1.9
Fecha:	Marzo 2023